

Project 6: CI/CD Pipeline for a Flask Application

Complete step-by-step implementation guide (AWS + GitHub Actions)

Architecture

Developer → GitHub Repository → GitHub Actions → Build & Test → Deploy → EC2 → Flask Application

Result: A code change pushed to GitHub automatically triggers testing and deployment, updating the Flask application running on EC2.

Step 1: Create a simple Flask application

Create a project folder with these files:

```
flask-cicd/  
■■■ app.py  
■■■ requirements.txt  
■■■ test_app.py  
■■■ .github/  
    ■■■ workflows/  
        ■■■ deploy.yml
```

app.py

```
from flask import Flask  
  
app = Flask(__name__)  
  
@app.route("/")  
def home():  
    return "Hello from Flask CI/CD!"  
  
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=5000)
```

requirements.txt

```
Flask  
pytest
```

test_app.py

```
from app import app  
  
def test_home():  
    client = app.test_client()  
    response = client.get("/")  
    assert response.status_code == 200  
    assert b"Hello from Flask CI/CD!" in response.data
```

Step 2: Create the GitHub repository

- Create a new GitHub repository, for example: flask-cicd.
- Push app.py, requirements.txt and test_app.py.
- Push the .github/workflows/deploy.yml file after creating the EC2 server and GitHub secrets.

Step 3: Launch an EC2 instance

- AWS Console → EC2 → Launch instance
- Name: Flask-CICD-Server
- AMI: Amazon Linux 2023
- Instance type: use the free-tier-eligible option shown in your AWS account
- Create/select a key pair
- Allow SSH (port 22) from your IP and Custom TCP port 5000 from Anywhere-IPv4 for exam testing.
- Launch instance.

Step 4: Prepare the EC2 server

Connect to EC2 using SSH, then run:

```
sudo dnf update -y  
sudo dnf install -y python3 git
```

```
sudo mkdir -p /opt/flask-cicd
sudo chown ec2-user:ec2-user /opt/flask-cicd
cd /opt/flask-cicd
```

Clone your GitHub repository once:

```
git clone https://github.com/YOUR_USERNAME/YOUR_REPO.git app
```

Then install dependencies:

```
cd /opt/flask-cicd/app
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Step 5: Create a systemd service for Flask

Create the service:

```
sudo nano /etc/systemd/system/flask-cicd.service
```

Paste:

```
[Unit]
Description=Flask CI/CD Application
After=network.target

[Service]
User=ec2-user
WorkingDirectory=/opt/flask-cicd/app
Environment="PATH=/opt/flask-cicd/app/venv/bin"
ExecStart=/opt/flask-cicd/app/venv/bin/python app.py
Restart=always

[Install]
WantedBy=multi-user.target
```

Start the application:

```
sudo systemctl daemon-reload
sudo systemctl enable flask-cicd
sudo systemctl start flask-cicd
sudo systemctl status flask-cicd
```

Test in browser:

```
http://EC2_PUBLIC_IP:5000
```

Step 6: Configure GitHub Actions deployment

Create `.github/workflows/deploy.yml` in the repository:

```
name: Flask CI/CD

on:
  push:
    branches: [ "main" ]

jobs:
  test-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run tests
        run: pytest

      - name: Deploy to EC2
        uses: appleboy/ssh-action@v1.2.0
        with:
          host: ${ secrets.EC2_HOST }
          username: ec2-user
          key: ${ secrets.EC2_SSH_KEY }
          script: |
            cd /opt/flask-cicd/app
            git pull origin main
            source venv/bin/activate
            pip install -r requirements.txt
            sudo systemctl restart flask-cicd
            sudo systemctl status flask-cicd --no-pager
```

Step 7: Add GitHub Secrets

GitHub repository → Settings → Secrets and variables → Actions → New repository secret

Secret	Value
EC2_HOST	EC2 public IPv4 address
EC2_SSH_KEY	Complete private key (.pem) contents

Important: never commit the `.pem` private key to GitHub. Store it only as the GitHub Actions secret.

Step 8: Push the workflow

```
git add .
git commit -m "Add Flask CI/CD pipeline"
git push origin main
```

GitHub → Actions tab → open the workflow run. It should show checkout → install → test → deploy.

Step 9: Demonstrate automatic deployment

Change the response in `app.py`, for example:

```
return "Version 2 - Deployed automatically!"
git add app.py
git commit -m "Update Flask application"
git push origin main
```

GitHub Actions automatically runs the tests and, if they pass, connects to EC2, pulls the new code, installs dependencies and restarts Flask.

Step 10: Final exam demonstration

1. Open the EC2 application URL and show the original Flask response.
 2. Show the GitHub repository and the Actions workflow.
 3. Change the Flask message and push the change.
 4. Open GitHub → Actions and show the workflow running.
 5. Show the test step as successful.
 6. Show the deployment step as successful.
 7. Refresh the EC2 URL and show the updated message.
- This proves automated **build/test/deployment** after a source-code change.

Quick architecture for viva

Code Push → GitHub → GitHub Actions → Install → Test → SSH to EC2 → git pull → Restart Flask → Updated Application

Cleanup

After the exam, stop/terminate the EC2 instance if it is no longer required. Remove unused AWS resources to avoid charges.